

Sprint 2 — Refatoração de `teams.py`

Objetivo

A Sprint 2 deve reorganizar `teams.py` para separar claramente três responsabilidades: montagem do contexto da página, renderização das tabs de leitura e fluxo administrativo de `transactions`.[\[file:683\]](#) Hoje a página mistura carregamento de dados, cálculos de roster, `summary cards`, montagem de `picks`, histórico de `transactions` e formulário de cadastro em um único arquivo longo, o que dificulta manutenção e evolução.[\[file:683\]](#)

O código atual já possui uma boa base de domínio fora da UI: `transforms.py` centraliza montagem de roster, `picks`, totais e sumários; `transactions_service.py` já concentra validação, histórico, `append` e `update` de rosters/`picks`.[\[file:694\]](#)[\[file:693\]](#) A Sprint 2 deve aproveitar isso e mover o que ainda está acoplado à página para `helpers` e módulos de UI específicos.[\[file:683\]](#)[\[file:693\]](#)[\[file:694\]](#)

Diagnóstico atual

Em `teams.py`, a página hoje acumula responsabilidades demais em sequência: autenticação local, cache, carregamento do workbook, seleção de `time/temporada`, construção de lookups, cálculo de rosters MAIN/DEV, transformação de `picks`, cálculo de `cap`, `summary cards`, tabs e um bloco extenso de `transactions`.[\[file:683\]](#) Esse desenho torna difícil testar pedaços isolados e amplia o risco de regressão quando o fluxo administrativo muda.[\[file:683\]](#)

Há também duplicação de lógica ou reaproveitamento incompleto. O código primeiro usa `build_picks_view()` e `format_picks_for_display()`, mas depois sobrescreve `picks_display` com um `copy()` cru de `team_picks_df`, reconstruindo manualmente colunas e labels.[\[file:683\]](#)[\[file:694\]](#) Isso é um sinal claro de que falta uma camada de “page context” bem definida para abastecer a UI com objetos já preparados.[\[file:683\]](#)[\[file:694\]](#)

O trecho de `transactions` é o maior ponto de complexidade. Embora já exista avanço com os tipos `TRADE`, `WAIVE`, `ADD`, `MOVE`, o arquivo ainda concentra escolha de times, escolha de assets, regras visuais por tipo, preparação de payload e acionamento de persistência no mesmo bloco renderizado dentro da aba.[\[file:683\]](#) Isso dificulta ajustes futuros, especialmente porque `validate_items_bilateral()` e `update_rosters()` já trazem regras de domínio que a UI deveria apenas orquestrar.[\[file:693\]](#)[\[file:683\]](#)

O que manter

Algumas partes estão bem posicionadas e devem ser preservadas como núcleo funcional da página. `transforms.py` já fornece `get_team_options()`, `get_visible_seasons()`, `build_roster_view()`, `format_roster_for_display()`, `calculate_main_totals()`, `calculate_dev_totals()`, `summarize_positions()` e `summarize_picks_by_year()`, cobrindo o essencial da camada de transformação.[\[file:694\]](#)

Em `transactions`, `transactions_service.py` já encapsula elementos importantes de domínio: domínio de `players` e `picks` por `time`, validação bilateral, `append` das transações, atualização de `rosters/picks` e montagem do histórico consolidado.[file:693] A Sprint 2 não deve reescrever esse `service` agora; deve apenas criar uma camada de UI mais limpa sobre ele.[file:693]

Estrutura proposta

A proposta para a Sprint 2 é quebrar `teams.py` em uma composição simples:

Arquivo	Responsabilidade principal
<code>pages/teams.py</code>	Página fina, orquestra login, carga do contexto e render das seções.[file:683]
<code>app_lib/teams_page_context.py</code>	Monta todo o contexto da página: <code>time</code> selecionado, <code>rosters</code> , <code>totais</code> , <code>picks</code> , <code>status</code> do <code>cap</code> , histórico e <code>lookups</code> .[file:683][file:694][file:693]
<code>app_lib/teams_tabs_ui.py</code>	Renderiza tabs de leitura: <code>MAIN</code> , <code>DEV</code> , <code>Picks</code> e histórico de <code>transactions</code> .[file:683]
<code>app_lib/transactions_ui.py</code>	Renderiza o formulário administrativo de <code>transactions</code> e devolve <code>payload</code> validado para persistência.[file:683][file:693]
<code>app_lib/transactions_form_helpers.py</code>	Helpers puros para normalizar <code>times</code> , decidir blocos visíveis e preparar <code>items/tx row</code> .[file:683][file:693]

Essa divisão respeita o desenho já existente do projeto, que usa `app_lib` para serviços e `transforms` e deixa a página como ponto de entrada.[file:683][file:694][file:693]

Ordem de execução

A implementação da Sprint 2 deve seguir esta ordem:

1. Criar `teams_page_context.py`.
2. Extrair render das tabs de leitura para `teams_tabs_ui.py`.
3. Extrair o formulário de `transactions` para `transactions_ui.py`.
4. Extrair helpers puros do formulário para `transactions_form_helpers.py`.
5. Enxugar `teams.py` para virar apenas composição final.[file:683][file:693][file:694]

Essa ordem é a mais segura porque primeiro estabiliza os dados exibidos na página, depois separa a leitura da escrita, e só por último simplifica o `entrypoint`.[file:683][file:693][file:694]

Passo 1 — `teams_page_context.py`

Esse módulo deve concentrar toda a preparação de dados da página, reduzindo o topo de `teams.py` a poucas linhas. O arquivo atual calcula manualmente `team_lookup`, `player_lookup`, `main_roster`, `dev_roster`, `main_totals`, `dev_totals`, `team_picks_df`, `pick_year_counts`, `team_transactions_df`, `cap_remaining` e `cap_status` no corpo principal da página.[file:683]

A proposta é criar uma função como `build_teams_page_context(data, selected_team_name, selected_start_season) -> dict`, devolvendo um dicionário único com tudo que a UI precisa.[file:683][file:694][file:693] Esse contexto deve incluir pelo menos:

- `selected_team_id, selected_team_name, visible_seasons.`[file:683][file:694]
- `team_lookup, player_lookup.`[file:683]
- `main_roster, dev_roster, display_main, display_dev.`[file:683][file:694]
- `main_totals, dev_totals, main_summary, cap_remaining, cap_status.`[file:683][file:694]
- `team_picks_df, picks_display, pick_year_counts.`[file:683][file:694]
- `team_transactions_df.`[file:683][file:693]
- `main_positions_text, dev_positions_text, total_picks.`[file:683]

Também vale corrigir nesse módulo a inconsistência de picks. Hoje `picks_display` é criado formatado e depois substituído por um `copy()` de `team_picks_df`, com renomeações manuais. [file:683] O contexto deve definir uma única fonte de verdade para a aba de picks, evitando dupla montagem.[file:683][file:694]

Assinatura sugerida

```
# app_lib/teams_page_context.py
from __future__ import annotations

import pandas as pd
from app_lib.transforms import (
    get_team_options,
    get_visible_seasons,
    build_roster_view,
    format_roster_for_display,
    calculate_main_totals,
    calculate_dev_totals,
    summarize_positions,
    summarize_picks_by_year,
    build_picks_view,
    format_picks_for_display,
)
from app_lib.transactions_service import (
    TX_SHEET,
    TX_ITEMS_SHEET,
    build_transactions_history,
)
from app_lib.teams_ui_helpers import build_red_flags

def build_teams_page_context(data: dict, selected_team_name: str, selected_start_season: int, ...)
```

Passo 2 — `teams_tabs_ui.py`

As tabs Principal, Development, Picks e a parte de histórico em Transactions devem sair de `teams.py` e virar funções pequenas de render.[file:683] Hoje essas tabs já recebem dados praticamente prontos; portanto a extração é de baixo risco se o contexto vier bem montado. [file:683]

A ideia é criar funções como:

- `render_main_tab(ctx);` [file:683]
- `render_dev_tab(ctx);` [file:683]
- `render_picks_tab(ctx);` [file:683]
- `render_transactions_history_tab(ctx)` ou `render_transactions_history(df).` [file:683] [file:693]

Isso reduz o acoplamento visual e facilita futuras mudanças de layout, porque o comportamento de exibição deixa de competir no mesmo arquivo com o comportamento de formulário e persistência.[file:683]

Exemplo de interface

```
# app_lib/teams_tabs_ui.py
import streamlit as st
from app_lib.teams_ui_helpers import (
    currency,
    get_roster_column_order,
    get_roster_column_config,
    get_picks_column_order,
    get_picks_column_config,
)

def render_main_tab(ctx: dict):
    ...

def render_dev_tab(ctx: dict):
    ...

def render_picks_tab(ctx: dict):
    ...

def render_transactions_history(ctx: dict):
    ...
```

Passo 3 — transactions_form_helpers.py

Antes de mover o formulário inteiro, vale extrair a parte que não precisa de Streamlit. O trecho atual tem decisões repetidas sobre visibilidade de Time B, normalização de `from_team_id/to_team_id`, cálculo de `same_team`, defaults de contagem e preparação de `prepared_items`. [file:683]

Essas regras devem virar helpers puros, por exemplo:

- `normalize_team_ids(tx_type, from_team_name, to_team_name, team_name_to_id).` [file:683]

- `get_transaction_mode_flags(tx_type)` para retornar `show_to_team`, `is_move`, `is_trade`, `is_add`, `is_waive`.[\[file:683\]](#)
- `build_tx_row_payload(...)`.[\[file:683\]](#)
- `build_transaction_item_payloads(...)`.[\[file:683\]](#)
- `validate_basic_transaction_form(...)` para validações de tela antes de chamar `validate_items_bilateral()`.[\[file:683\]](#)[\[file:693\]](#)

Com isso, a UI deixa de ter regras de montagem misturadas com widgets e o service continua dono da validação de domínio mais sensível.[\[file:683\]](#)[\[file:693\]](#)

Exemplo de helper útil

```
# app_lib/transactions_form_helpers.py

def normalize_team_ids(tx_type: str, from_team_name: str | None, to_team_name: str |
    from_team_id = team_name_to_id.get(from_team_name) if from_team_name else None
    to_team_id = team_name_to_id.get(to_team_name) if to_team_name else None

    tx_type = str(tx_type).strip().upper()

    if tx_type == "WAIVE":
        to_team_id = None
    elif tx_type == "ADD":
        from_team_id = None
    elif tx_type == "MOVE":
        to_team_id = from_team_id

    return from_team_id, to_team_id
```

Passo 4 — `transactions_ui.py`

Depois que os helpers puros existirem, o formulário administrativo pode sair de `teams.py` quase inteiro. Esse módulo deve renderizar a expande de cadastro, usar os helpers de normalização e, ao final, devolver uma ação concluída ou não.[\[file:683\]](#)

A função central pode ser algo como `render_transactions_admin_form(data, teams, user, default_file) -> bool`, retornando `True` quando uma transaction for salva com sucesso, para que a página principal apenas limpe cache e rode `st.rerun()`.[\[file:683\]](#) Outra opção ainda melhor é a própria função já executar `append_transaction()`, `update_rosters()`, limpar cache e `rerun`, deixando `teams.py` sem qualquer regra de escrita.[\[file:683\]](#)[\[file:693\]](#)

Esse módulo deve conter:

- widgets do cabeçalho da transaction; [\[file:683\]](#)
- render condicional dos blocos de assets por tipo; [\[file:683\]](#)
- chamada de `validate_items_bilateral(data, valid_item_rows, tx_type)`; [\[file:693\]](#)
- criação de `tx_row` e `prepared_items`; [\[file:683\]](#)
- persistência via `append_transaction()` e `update_rosters()`.[\[file:693\]](#)

Passo 5 — `teams.py` final

Ao fim da Sprint 2, `teams.py` deve ficar próximo deste formato conceitual:

```
from pathlib import Path
import streamlit as st

from app_lib.data_loader import load_workbook_data, SEASONS
from app_lib.role_helpers import is_admin_user
from app_lib.session_helpers import require_login_v5
from app_lib.teams_page_context import build_teams_page_context
from app_lib.teams_tabs_ui import (
    render_main_tab,
    render_dev_tab,
    render_picks_tab,
    render_transactions_history,
)
from app_lib.transactions_ui import render_transactions_admin_form
from app_lib.teams_ui_helpers import inject_summary_card_css, render_summary_card, c

DEFAULT_FILE = Path("roster.xlsx")

@st.cache_data
def cached_load(file_path: str):
    return load_workbook_data(file_path)

user = require_login_v5()
is_admin = is_admin_user(user)

data = cached_load(str(DEFAULT_FILE))
ctx = build_teams_page_context(data, selected_team_name, selected_start_season)

# summary cards
# tabs
# admin form
# history
```

Nesse ponto, a página vira apenas composição e navegação visual, que é exatamente o papel ideal dela.[file:683]

Ajustes recomendados já na Sprint 2

Além da extração estrutural, há alguns ajustes que valem entrar nesta sprint porque estão diretamente ligados ao refactor:

1. Consolidar picks em uma única representação

A aba de picks não deve alternar entre `format_picks_for_display()` e renomeações manuais sobre `team_picks_df`. [file:683][file:694] Defina se a UI quer um dataframe cru ou um dataframe de exibição e mantenha só uma trilha. [file:683][file:694]

2. Remover dependências não usadas

`teams.py` ainda importa `require_admin_page` e `admin_only_action`, mas o trecho atual usa apenas `is_admin_user()`.[\[file:683\]](#) Na Sprint 2, a limpeza dos imports deve acompanhar a redução do arquivo.[\[file:683\]](#)

3. Padronizar nomes de estado de transação

O formulário usa `TRADE`, `WAIVE`, `ADD`, `MOVE`, `OTHER`, enquanto o service aceita também aliases como `DISPENSA`, `DROP`, `SIGN`, `PROMOTION`.[\[file:683\]](#)[\[file:693\]](#) A UI deve trabalhar com um conjunto canônico e o service pode continuar tolerante por compatibilidade.[\[file:693\]](#)

4. Isolar preparação dos lookups

`team_lookup` e `player_lookup` devem nascer no contexto da página, e não no corpo de render principal.[\[file:683\]](#) Isso reduz ruído e facilita reuso por tabs e histórico.[\[file:683\]](#)[\[file:693\]](#)

Critério de pronto

A Sprint 2 pode ser considerada pronta quando estes pontos forem atendidos:

- `teams.py` deixa de concentrar cálculos de contexto e o formulário completo de transactions.[\[file:683\]](#)
- `teams_page_context.py` entrega um dicionário único e estável para a UI.[\[file:683\]](#)[\[file:694\]](#)[\[file:693\]](#)
- `teams_tabs_ui.py` renderiza as tabs de leitura sem lógica pesada embutida.[\[file:683\]](#)
- `transactions_ui.py` concentra o formulário admin de transactions.[\[file:683\]](#)[\[file:693\]](#)
- `transactions_form_helpers.py` concentra normalização e montagem de payloads.[\[file:683\]](#)[\[file:693\]](#)
- o fluxo `TRADE`, `WAIVE` e `MOVE` continua funcionando como hoje, mas com responsabilidades separadas.[\[file:683\]](#)[\[file:693\]](#)

Próxima ação prática

A melhor forma de avançar agora é começar por `teams_page_context.py`, porque ele reduz a complexidade de todo o resto e prepara a extração limpa das tabs e do formulário.[\[file:683\]](#)[\[file:694\]](#)[\[file:693\]](#) Depois dele, a próxima entrega deve ser `teams_tabs_ui.py`; só então vale mover o bloco de transactions para `transactions_ui.py`.[\[file:683\]](#)